

CONTENTS

Práctico 10 (Interrupciones y E/S en 8086) — Arquitectura de Computadoras	
FING UdelaR · Resolución completa y verificada	
Verificación aritmética previa (para todo el práctico)	
Preguntas teóricas	
Ejercicio 0 — Máquinas de estado (Práctico 9, Ej. 3, 6 y 7)	
Ejercicio 1 — Compilación a assembler 8086	
A) activarBomba() y desactivarBomba() (base: Práctico 9, Ej. 2)	
B) Instalación de la interrupción del teclado (n=4, rutina en 0x41024)	
Ejercicio 2 — nuevoChar (protocolo serial con canal)	
A) Rutina en C	
B) Compilación a assembler 8086	
Ejercicio 3 — Sistema de seguridad de gas (máquina dedicada)	
A) Rutinas en C	
B) Compilación a assembler 8086	
C) Instalación de la interrupción del reloj (n=9, rutina en 0x3340A)	
Ejercicio 4 — Tiro al blanco	
A) Diseño e implementación en C	
B) Compilación a assembler 8086	
C) Vector de interrupciones para timer (13) e interruptor (14), mismo CS	

Ejercicio 5 — Decodificador PWM (máquina dedicada)

A) Diseño e implementación en C

B) Instalación de flanco (15) y timer (16) en el segmento 0x5500

Rutinas en assembler 8086

Resumen de técnicas usadas en este práctico

Práctico 10 (Interrupciones y E/S en 8086) — Arquitectura de Computadoras

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

 **Nota general de transparencia:** la solución oficial de este práctico ([Ejercicios Resueltos.pdf](#)) solo cubre teoría, Ejercicio 0, 1, 2 y 3 — **no incluye** una resolución de los Ejercicios 4 y 5. Esos dos se resuelven acá desde cero. Además, se verificó **cada** dirección de vector de interrupciones ($n \times 4$) con una cuenta explícita, y se encontraron **4 errores reales** en el material oficial (2 ya conocidos de la guía temática de Interrupciones, y 2 adicionales nuevos, detallados donde corresponden). El Ejercicio 0 pide diagramar máquinas de estado del Práctico 9 a partir de figuras — se transcriben tal como aparecen en el material oficial (no involucran aritmética, así que no hay margen para el error de conversión decimal→hex que motivó este práctico).

Verificación aritmética previa (para todo el práctico)

La dirección de la entrada del vector de interrupciones para la interrupción **n** es siempre **$n \times 4$** , calculada primero en **decimal** y **recién después** convertida a hexadecimal (nunca pegar “0x” sobre el número decimal). Verificación explícita con Python de todas las interrupciones usadas en este práctico:

```
>>> for n in [4, 9, 13, 14, 15, 16]:
...     print(n, "-> offset:", hex(n*4), " segmento:", hex(n*4+2))
4  -> offset: 0x10  segmento: 0x12
9  -> offset: 0x24  segmento: 0x26
13 -> offset: 0x34  segmento: 0x36
14 -> offset: 0x38  segmento: 0x3a
15 -> offset: 0x3c  segmento: 0x3e
16 -> offset: 0x40  segmento: 0x42
```

Estos 6 valores se usan en los Ejercicios 1, 3, 4 y 5. Cualquier valor distinto a estos en la solución oficial es un error de conversión.

Preguntas teóricas

(a) **Ubicación del vector de interrupciones:** ocupa las direcciones absolutas **0x00000 a 0x003FF** (256 entradas $\times$ 4 bytes = 1024 bytes = 0x400). Es una posición **fija**, determinada por el diseño del hardware del procesador — no se puede reubicar.

(b) **Propósito:** almacenar, para cada uno de los 256 números de interrupción posibles, la dirección absoluta (segmento:desplazamiento) de la rutina de atención correspondiente, para que la CPU pueda saltar directamente a ella al recibir un pedido.

(c) **“Instalar la rutina” en 8086 implica:** deshabilitar las interrupciones (`CLI`), calcular la posición $n \times 4$ dentro del vector para el número de interrupción asignado, y escribir ahí el desplazamiento (2 bytes) y, dos bytes más adelante ($n \times 4 + 2$), el segmento de la rutina — y luego rehabilitar interrupciones (`STI`). Se deshabilita durante la escritura para evitar que, si justo en ese momento llega un pedido de esa misma interrupción, la CPU lea una entrada del vector a medio escribir (offset ya actualizado pero segmento todavía viejo, o viceversa) y salte a una dirección inválida.

(d) **Recordar valores entre dos ejecuciones de la misma rutina:** usando **variables globales en memoria**, fuera del espacio de la pila — la pila se vacía en cada `IRET`, así que cualquier valor que deba sobrevivir entre invocaciones (como un contador de segundos) debe guardarse en una dirección de memoria fija y accesible por la rutina en cada llamada.

Ejercicio 0 — Máquinas de estado (Práctico 9, Ej. 3, 6 y 7)

⚠ Estos diagramas provienen de figuras del Práctico 9 que no están a la vista en este documento — se transcriben tal como constan en el material oficial de referencia. No involucran cálculo de direcciones, por lo que no aplica el chequeo de $n \times 4$.

Ejercicio 3 del P9 (intérprete de comandos por 3 teclas + ESC): tres estados —

`ESPERANDO_ESC`, `ESPERANDO_NUM`, `EJECUTANDO_RUT`.

```
ESPERANDO_ESC --(ESC)--> ESPERANDO_NUM
ESPERANDO_ESC --(!=ESC)--> ESPERANDO_ESC      (se mantiene)
ESPERANDO_NUM --(ESC)--> ESPERANDO_NUM        (se mantiene)
ESPERANDO_NUM --(!=1,2,3,ESC)--> ESPERANDO_ESC
ESPERANDO_NUM --(1,2,3)--> EJECUTANDO_RUT
EJECUTANDO_RUT --(!=ESC)--> EJECUTANDO_RUT    (se mantiene)
EJECUTANDO_RUT --(ESC)--> ESPERANDO_ESC
```

Ejercicio 6 del P9 (pasillo con luces, ver también sección 8 de la guía temática de Interrupciones): dos estados — `APAGADA`, `ENCENDIDA`.

```
APAGADA --(boton())--> ENCENDIDA
ENCENDIDA --(tiempo() & seg==0)--> APAGADA
```

Ejercicio 7 del P9 (control de fuga de gas — base del Ejercicio 3 de este práctico): cuatro estados — `ABIERTA_SIN_FUGA`, `ABIERTA_CON_FUGA`, `CERRADA`, `REINICIAR`.

```
ABIERTA_SIN_FUGA --(SENSOR&0x01==1)--> ABIERTA_CON_FUGA
ABIERTA_CON_FUGA --(SENSOR&0x01==0 & seg<=300)--> ABIERTA_SIN_FUGA
ABIERTA_CON_FUGA --(seg>300)--> CERRADA
CERRADA --(SENSOR&0x01==0)--> REINICIAR
```

Nota de consistencia: `seg>300` con un reloj de 10 Hz corresponde a **30 segundos** (10 pedidos/segundo $\times$ 30 s = 300 pedidos) — coincide exactamente con el enunciado del Ejercicio 3 de este práctico (“si el escape persiste por más de 30 segundos”).

Ejercicio 1 — Compilación a assembler 8086

A) `ACTIVARBOMBA()` Y `DEACTIVARBOMBA()` (BASE: PRÁCTICO 9, EJ. 2)

```

BOMBA equ 0x123

activarBomba proc
    push ax
    push dx

    mov dx, BOMBA
    in  al, dx
    or  al, 0x04      ; activo la señal de arranque de la bomba
    out dx, al

_esperar_lista:
    in  al, dx
    and al, 0x01      ; bit de "bomba lista"
    jz  _esperar_lista ; espero activamente a que la bomba esté lista

    in  al, dx
    or  al, 0x10      ; habilito el flujo, ya con la bomba lista
    out dx, al

    pop dx
    pop ax
    ret
activarBomba endp

desactivarBomba proc
    push ax
    push dx

    mov dx, BOMBA
    in  al, dx
    and al, 0xFB      ; corto la señal de arranque
    out dx, al

_esperar_detenida:
    in  al, dx
    and al, 0x01
    jnz _esperar_detenida ; espero a que la bomba deje de estar "lista/activa"

    in  al, dx
    and al, 0xEF      ; cierro el flujo
    out dx, al

    pop dx
    pop ax

```

```
ret
desactivarBomba endp
```

Verificación PUSH/POP: ambas rutinas hacen `push ax; push dx` al entrar y `pop dx; pop ax` antes de salir — orden exactamente inverso (LIFO), balanceado 2 PUSH ↔ 2 POP. Como no son rutinas instaladas en el vector de interrupciones (se invocan con `call` desde otra rutina, no directamente desde `INT n`), terminan en `ret`, no en `iret` — correcto.

B) INSTALACIÓN DE LA INTERRUPCIÓN DEL TECLADO (N=4, Rutina en 0x41024)

Paso 1 — descomponer la dirección física en segmento:desplazamiento. DIR = SEGMENTO×16 + DESPLAZAMIENTO. Elijo SEGMENTO=0x4000, DESPLAZAMIENTO=0x1024: $0x4000 \times 16 + 0x1024 = 0x40000 + 0x1024 = \mathbf{0x41024}$ ✓

Paso 2 — posición en el vector: $n \times 4 = 4 \times 4 = \mathbf{16 \text{ decimal}} = \mathbf{0x10 \text{ hexadecimal}}$ (verificación por corrimiento: $0x04 \ll 2 = 0x10$ ✓). El segmento va 2 bytes más adelante: $0x10 + 2 = \mathbf{0x12}$.

```
cli
mov ax, 0
mov es, ax
mov word ptr es:[0x10], 0x1024 ; offset de la rutina de teclado
mov word ptr es:[0x12], 0x4000 ; segmento de la rutina de teclado
sti
```

Error real corregido (ya detectado al armar la guía temática): la solución oficial de este ejercicio escribe `es:[0x16]` y `es:[0x18]`. Es un error de conversión: $4 \times 4 = 16$ en **decimal**, pero 16 decimal no es “0x16” — convertido correctamente a hexadecimal, 16 decimal es **0x10**. El error consiste en pegarle el prefijo “0x” directamente al resultado decimal (16 → “0x16”) en lugar de convertirlo. Corregido arriba a `es:[0x10]` / `es:[0x12]`.

Ejercicio 2 — `nuevoChar` (protocolo serial con canal)

A) RUTINA EN C

```

#define NO_MSG    0
#define ESP_CANAL 1
#define EN_MSG    2

// Variables globales (para recordar el estado entre invocaciones – ver Teórica (d)):
char car[1024];
int  pos = 0;
int  estado = NO_MSG;

void nuevoChar() {
    char letra = in(MENSAJE);

    if (estado == NO_MSG) {
        if (letra == STX)
            estado = ESP_CANAL;
        // si no es STX, se ignora: seguimos esperando el inicio de mensaje
    }
    else if (estado == ESP_CANAL) {
        if ((letra > 0) && (letra <= 10))
            estado = EN_MSG;          // canal válido (1..10): empieza el texto
        else
            estado = NO_MSG;          // canal 0 o inválido: se descarta el mensaje com-
pleto
    }
    else { // estado == EN_MSG
        if (letra == ETX)
            estado = NO_MSG;          // fin de mensaje
        else {
            car[pos] = letra;
            pos = (pos + 1) % 1024;    // cola circular de 1024 caracteres
        }
    }
}

```

Error adicional encontrado (no reportado antes): la solución oficial declara la variable global como `int carPos = 0;` pero el cuerpo de la función usa `car[pos]` y `pos = (pos+1)%1024` — es decir, usa el identificador `pos`, que nunca fue declarado, mientras que `carPos` queda declarado y sin usar. Tal cual está en el material oficial, ese código de C no compila. Corregido arriba unificando el nombre de la variable a `pos` en la declaración y en el cuerpo.

Nota de diseño: el canal 0 se descarta explícitamente (el enunciado pide guardar “los caracteres de los mensajes con canal diferente a 0”); por eso si `estado==ESP_CANAL` y `letra==0`, se vuelve a `NO_MSG` sin pasar nunca a `EN_MSG`.

B) COMPILACIÓN A ASSEMBLER 8086

Dato de layout: la cola circular arranca en `ES:[2]`, y `ES:[0]` guarda la cantidad de caracteres ya almacenados (puntero de escritura circular).

```

estado dw 0
pos     dw 0
MENSAJE equ 0x123

nuevoChar proc far
    push ax
    push dx
    push bx

    mov dx, MENSAJE
    in  al, dx                ; al = carácter recibido

    cmp word ptr [estado], 0    ; NO_MSG
    jne _elseif_espcanal
    cmp al, STX
    jne _fin
    mov word ptr [estado], 1    ; -> ESP_CANAL
    jmp _fin

_elseif_espcanal:
    cmp word ptr [estado], 1    ; ESP_CANAL
    jne _else_enmsg
    cmp al, 0
    jle _canal_invalido
    cmp al, 10
    jg  _canal_invalido
    mov word ptr [estado], 2    ; -> EN_MSG (canal válido 1..10)
    jmp _fin

_canal_invalido:
    mov word ptr [estado], 0    ; -> NO_MSG (se descarta)
    jmp _fin

_else_enmsg:
    ; estado == EN_MSG
    cmp al, ETX
    jne _guardar_char
    mov word ptr [estado], 0    ; fin de mensaje -> NO_MSG
    jmp _fin

_guardar_char:
    mov bx, es:[0]              ; bx = posición actual en la cola
    mov es:[bx+2], al           ; guardo el carácter (cola arranca en ES:[2])
    inc bx
    cmp bx, 1024
    jl  _guardar_pos
    sub bx, 1024                ; wraparound circular
_guardar_pos:
    mov es:[0], bx

_fin:
    pop bx

```

```

    pop dx
    pop ax
    iret
nuevoChar endp

```

Verificación PUSH/POP: 3 PUSH (ax, dx, bx) al entrar ↔ 3 POP (bx, dx, ax) antes de retornar, orden exactamente inverso. Balanceado.

Error adicional encontrado (no reportado antes): la solución oficial declara `nuevoChar` `proc` (sin `far`) y termina con `ret`. Como `nuevoChar` es la rutina que se invoca directamente al recibir un carácter (es decir, está **instalada en el vector de interrupciones**, igual que `tiempo()` en el Ejercicio 3, que sí usa `iret`), debe ser un procedimiento `far` (para que el desplazamiento y segmento completos sean válidos sin importar desde qué CS se la invoque) y debe retornar con `IRET` —no `RET`— porque debe restaurar también CS, IP y los **flags** (incluyendo `IF`, que el hardware había puesto en 0 al entrar) que el hardware apiló automáticamente al procesar la interrupción. Un `RET` en su lugar dejaría el flag `IF` en 0 para siempre (las interrupciones quedarían deshabilitadas permanentemente tras la primera invocación) y no restauraría CS correctamente. Corregido arriba: `proc far ... iret`.

Ejercicio 3 — Sistema de seguridad de gas (máquina dedicada)

A) RUTINAS EN C

Usando los 4 estados verificados en el Ejercicio 0 (`SIN_FUGA`, `CON_FUGA`, `CERRADA`, `REINICIADO`) y el reloj de 10 Hz atendido por `tiempo()`. 300 tics de 10 Hz = 30 s (umbral del enunciado).

```

#define SIN_FUGA    0
#define CON_FUGA    1
#define CERRADA     2
#define REINICIADO  3

int estado = SIN_FUGA;
int tics = 0;

void tiempo() {
    char sensor = in(SENSOR) & 0x01;    // 1 = concentración peligrosa
    char accesorio = in(ACCESORIO);

    switch (estado) {
        case SIN_FUGA:
            if (sensor == 1) {
                estado = CON_FUGA;
                tics = 0;
                accesorio = accesorio | 0x01;    // enciendo extractor (bit 0)
            }
            break;

        case CON_FUGA:
            tics = tics + 1;
            if (tics > 300) {                // fuga > 30 s
                estado = CERRADA;
                accesorio = accesorio & 0xFD;    // cierro válvula (bit 1 = 0); extrac-
tor sigue prendido
            } else if (sensor == 0) {
                estado = SIN_FUGA;
                accesorio = accesorio & 0xFE;    // se terminó la fuga a tiempo: apago
extractor
            }
            break;

        case CERRADA:
            if (sensor == 0) {
                estado = REINICIADO;
                accesorio = accesorio & 0xFE;    // fuga terminada: apago extractor
(válvula sigue cerrada)
            }
            break;

        case REINICIADO:
            break;    // el sistema queda inactivo; la válvula solo se reabre manualmente
    }

    out(ACCESORIO, accesorio);
}

```

B) COMPILACIÓN A ASSEMBLER 8086

```

SENSOR      equ 0x1234
ACCESORIO   equ 0x5678

estado dw 0      ; 0=SIN_FUGA, 1=CON_FUGA, 2=CERRADA, 3=REINICIADO
tics      dw 0

tiempo proc far
    push ax
    push dx

    mov dx, SENSOR
    in  al, dx
    and al, 0x01
    mov ah, al          ; ah = bit de sensor (0/1)

    mov dx, ACCESORIO
    in  al, dx          ; al = estado actual de extractor/válvula

    cmp word ptr [estado], 0          ; SIN_FUGA
    jne _elif_confuga
    cmp ah, 1
    jne _salida
    mov word ptr [estado], 1
    mov word ptr [tics], 0
    or  al, 0x01
    jmp _salida

_elif_confuga:
    cmp word ptr [estado], 1          ; CON_FUGA
    jne _elif_cerrada
    inc word ptr [tics]
    cmp word ptr [tics], 300
    jle _check_fin_fuga
    mov word ptr [estado], 2          ; -> CERRADA
    and al, 0xFD
    jmp _salida
_check_fin_fuga:
    cmp ah, 0
    jne _salida
    mov word ptr [estado], 0          ; -> SIN_FUGA
    and al, 0xFE
    jmp _salida

_elif_cerrada:
    cmp word ptr [estado], 2          ; CERRADA (si no, es REINICIADO: no hace nada)
    jne _salida
    cmp ah, 0
    jne _salida

```

```

    mov word ptr [estado], 3          ; -> REINICIADO
    and al, 0xFE

_salida:
    mov dx, ACCESORIO
    out dx, al

    pop dx
    pop ax
    iret
tiempo endp

```

Verificación PUSH/POP: push ax, push dx ↔ pop dx, pop ax — balanceado, orden inverso, termina en `iret` (correcto: es la rutina instalada en el vector para la interrupción del reloj).

C) INSTALACIÓN DE LA INTERRUPCIÓN DEL RELOJ (N=9, RUTINA EN 0X3340A)

Paso 1 — segmento:desplazamiento. Elijo SEGMENTO=0x3000, DESPLAZAMIENTO=0x340A: $0x3000 \times 16 + 0x340A = 0x30000 + 0x340A = \mathbf{0x3340A}$ ✓

Paso 2 — posición en el vector: $n \times 4 = 9 \times 4 = \mathbf{36 \text{ decimal}} = \mathbf{0x24 \text{ hexadecimal}}$ (verificación: $0x09 \ll 2 = 0x24$ ✓). Segmento en $0x24+2 = \mathbf{0x26}$.

```

cli
mov ax, 0
mov es, ax
mov word ptr es:[0x24], 0x340A    ; offset de tiempo()
mov word ptr es:[0x26], 0x3000    ; segmento de tiempo()
sti

```

Error real corregido (ya detectado al armar la guía temática): la solución oficial escribe `es:[0x36]` y `es:[0x38]`. Error de conversión: $9 \times 4 = 36$ en **decimal**, que convertido a hexadecimal es **0x24**, no “0x36” (que sería pegar el “0x” sobre el 36 decimal). Corregido arriba a `es:[0x24]` / `es:[0x26]`.

Ejercicio 4 — Tiro al blanco

⚠ Este ejercicio no está resuelto en el material oficial de referencia (el PDF de soluciones termina en el Ejercicio 3) — se resuelve íntegramente acá.

A) DISEÑO E IMPLEMENTACIÓN EN C

Dos estados: `IDLE` (esperando que alguien empiece el juego) y `JUGANDO`. El timer de 50 ms se usa como base de tiempo para dos cosas: avanzar la lámpara cada 100 ms (2 tics) y contar segundos hasta el límite de 900 s.

```

#define IDLE      0
#define JUGANDO   1
#define LAMPARAS  0x11
#define BOTON     0x10
#define RESULTADO 0x34008    // dirección de memoria absoluta

int estado = IDLE;
int lampaActual = 0;
unsigned char encendidas = 0x00; // bitmask de lámparas ya "ganadas" (fijas)
int contLamp = 0; // tics de 50 ms hasta completar 100 ms (avance de lámpara)
int contSeg = 0; // tics de 50 ms hasta completar 1 s
int segundos = 0; // segundos transcurridos desde que arrancó la partida

void timer() {
    if (estado != JUGANDO) return;

    contSeg = contSeg + 1;
    if (contSeg == 20) { // 20 * 50ms = 1s
        contSeg = 0;
        segundos = segundos + 1;
        if (segundos >= 900) {
            *(unsigned int far *) RESULTADO = 0xB0B0; // no se logró a tiempo
            estado = IDLE;
            out(LAMPARAS, encendidas); // apago la lámpara "móvil"; quedan solo las
fijas
            return;
        }
    }

    contLamp = contLamp + 1;
    if (contLamp == 2) { // 2 * 50ms = 100ms
        contLamp = 0;
        lampaActual = (lampaActual + 1) % 8;
        out(LAMPARAS, encendidas | (1 << lampaActual));
    }
}

void interruptor() {
    char boton = in(BOTON);

    if (estado == IDLE) {
        estado = JUGANDO;
        encendidas = 0x00;
        lampaActual = 0;
        contLamp = 0;
        contSeg = 0;
        segundos = 0;
        out(LAMPARAS, 1 << lampaActual);
    } else {
        if (boton == lampaActual) {

```



```

    encendidas = encendidas | (1 << lampaActual);
    out(LAMPARAS, encendidas | (1 << lampaActual));
    if (encendidas == 0xFF) {
        *(unsigned int far *) RESULTADO = segundos;
        estado = IDLE;
    }
}
// si boton != lampaActual: fallo silencioso, el juego sigue
}
}

```

Nota de transparencia: el enunciado no especifica en qué unidad debe guardarse “el tiempo empleado” en 0x34008. Se optó por segundos enteros (contados con precisión de 50 ms vía `timer()`), que es la unidad en la que está expresado el resto del enunciado (900 segundos). Es una decisión de diseño razonable, no un dato dado explícitamente.

B) COMPILACIÓN A ASSEMBLER 8086

```

estado      dw 0
lampaAct    dw 0
encendidas  db 0
contLamp     dw 0
contSeg      dw 0
segundos    dw 0

LAMPARAS equ 0x11
BOTON      equ 0x10

timer proc far
    push ax
    push bx
    push cx

    cmp word ptr [estado], 1      ; JUGANDO?
    jne _fin

    inc word ptr [contSeg]
    cmp word ptr [contSeg], 20
    jne _avanzar_lamp
    mov word ptr [contSeg], 0
    inc word ptr [segundos]
    cmp word ptr [segundos], 900
    jle _avanzar_lamp

    ; se acabó el tiempo sin completar el juego
    mov word ptr [estado], 0
    mov ax, 0
    mov es, ax
    mov word ptr es:[0x34008], 0xB0B0
    mov al, byte ptr [encendidas]
    out LAMPARAS, al
    jmp _fin

_avanzar_lamp:
    inc word ptr [contLamp]
    cmp word ptr [contLamp], 2
    jne _fin
    mov word ptr [contLamp], 0
    mov bx, word ptr [lampaAct]
    inc bx
    and bx, 7                      ; (lampaActual+1) % 8
    mov word ptr [lampaAct], bx

    mov cl, bl
    mov ax, 1
    shl ax, cl                     ; ax = 1 << lampaActual

```

```

    mov bl, byte ptr [encendidas]
    or  al, bl
    out LAMPARAS, al

_fin:
    pop cx
    pop bx
    pop ax
    iret
timer endp

interruptor proc far
    push ax
    push bx
    push cx

    in  al, BOTON                ; al = último botón presionado (0..7)

    cmp word ptr [estado], 0     ; IDLE?
    jne _jugando

    ; arranca el juego
    mov word ptr [estado], 1
    mov byte ptr [encendidas], 0
    mov word ptr [lampaAct], 0
    mov word ptr [contLamp], 0
    mov word ptr [contSeg], 0
    mov word ptr [segundos], 0
    mov al, 1
    out LAMPARAS, al
    jmp _fin_int

_jugando:
    mov bl, al                  ; bl = botón presionado
    cmp bx, word ptr [lampaAct]
    jne _fin_int                ; falló: no pasa nada

    mov cl, byte ptr [lampaAct]
    mov ah, 1
    shl ah, cl                  ; ah = 1 << lampaActual
    or  byte ptr [encendidas], ah
    mov al, byte ptr [encendidas]
    out LAMPARAS, al

    cmp byte ptr [encendidas], 0xFF
    jne _fin_int
    ; ¡completó las 8 lámparas!
    mov word ptr [estado], 0
    push dx
    mov dx, 0
    mov es, dx

```

```

    mov ax, word ptr [segundos]
    mov word ptr es:[0x34008], ax
    pop dx

_fin_int:
    pop cx
    pop bx
    pop ax
    iret
interruptor endp

```

Verificación PUSH/POP: `timer`: push ax,bx,cx ↔ pop cx,bx,ax (balanceado, inverso).
`interruptor`: push ax,bx,cx (más un push/pop dx anidado y balanceado alrededor del bloque que escribe en 0x34008) ↔ pop cx,bx,ax en el orden correcto al final. Ambas terminan en `iret` (son rutinas instaladas en el vector).

C) VECTOR DE INTERRUPCIONES PARA TIMER (13) E INTERRUPTOR (14), MISMO CS

Posiciones en el vector: $13 \times 4 = 52$ decimal = **0x34 hex** (offset), 0x36 (segmento). $14 \times 4 = 56$ decimal = **0x38 hex** (offset), 0x3A (segmento). Verificación por corrimiento: $0x0D \ll 2 = 0x34$ ✓, $0x0E \ll 2 = 0x38$ ✓.

Elegir un segmento común para 0x55900 y 0x5600C: tomando SEGMENTO=0x5000:

```

>>> seg = 0x5000
>>> d1 = 0x55900 - seg*16 # -> 0x5900
>>> d2 = 0x5600C - seg*16 # -> 0x600C
>>> seg*16 + d1, seg*16 + d2
(0x55900, 0x5600c) # ambos verifican, y ambos desplazamientos caben en 16 bits
(<=0xFFFF)

```

```

cli
mov ax, 0
mov es, ax
mov word ptr es:[0x34], 0x5900 ; offset de timer (interrupción 13)
mov word ptr es:[0x36], 0x5000 ; segmento de timer
mov word ptr es:[0x38], 0x600C ; offset de interruptor (interrupción 14)
mov word ptr es:[0x3A], 0x5000 ; mismo segmento que timer
sti

```

Ejercicio 5 — Decodificador PWM (máquina dedicada)

⚠ Tampoco está resuelto en el material oficial — se resuelve íntegramente acá.

A) DISEÑO E IMPLEMENTACIÓN EN C

`flanco()` (por hardware, en cada flanco ascendente de PULSO) marca el **inicio** de un pulso;
`timer()` (1 kHz = cada 1 ms) hace de reloj para medir cuánto dura ese pulso, sondeando el bit de PULSO hasta detectar que bajó a 0 (fin del pulso). Umbral de decisión entre bit 0 (30 ms) y bit 1 (50 ms): el punto medio, 40 ms. Por debajo de 30 ms: marca (fin de byte).

```

#define PULSO    0x200    // dirección de E/S de solo lectura (bit 0 = señal)
#define SALIDA   0x201    // dirección de E/S de solo escritura

int contando = 0;          // 1 mientras se está midiendo el ancho de un pulso en
                           // curso
int anchoMs = 0;           // milisegundos transcurridos desde el flanco ascendente
int bitsAcum = 0;          // bits ya metidos en el byte actual (0..8)
unsigned char byteActual = 0; // byte en construcción (8 bits, con overflow natural)

void flanco() {
    contando = 1;
    anchoMs = 0;
}

void timer() {
    if (!contando) return;

    anchoMs = anchoMs + 1;
    if ((in(PULSO) & 0x01) != 0) return; // el pulso sigue activo, todavía no terminó

    contando = 0; // el pulso recién terminó: anchoMs mide su duración total

    if (anchoMs < 30) {
        // MARCA: cierro el byte actual
        while (bitsAcum < 8) { // relleno con 0 a la izquierda si faltan
            bits
                byteActual = byteActual << 1;
                bitsAcum = bitsAcum + 1;
            }
        out(SALIDA, byteActual);
        byteActual = 0;
        bitsAcum = 0;
    } else {
        int bit = (anchoMs < 40) ? 0 : 1; // <40ms ~ 30ms (bit 0); >=40ms ~ 50ms (bit
1)
        byteActual = (byteActual << 1) | bit; // el shift descarta solo el bit más viejo
si ya había 8
        if (bitsAcum < 8)
            bitsAcum = bitsAcum + 1;
        // si ya había 8 bits acumulados y llega un 9no antes de la marca,
        // "se toman los 8 últimos recibidos" – el shift de un byte de 8 bits
        // ya implementa esto automáticamente (el bit más antiguo se pierde).
    }
}

```

Nota de transparencia: el enunciado no da un umbral explícito entre 0 y 1 dentro del rango 30–50ms; se usó el punto medio (40ms) como frontera de decisión, que es el criterio estándar cuando solo se dan los dos valores extremos. La máquina es dedicada (lo dice el

enunciado), así que los registros usados en las rutinas podrían, por convención de diseño, no requerir guardado — aun así, en la versión assembler se los guarda con PUSH/POP para no depender de una convención no explicitada en el enunciado.

B) INSTALACIÓN DE `FLANCO` (15) Y `TIMER` (16) EN EL SEGMENTO 0X5500

Posiciones en el vector: $15 \times 4 = 60$ decimal = **0x3C** hex (offset), 0x3E (segmento). $16 \times 4 = 64$ decimal = **0x40** hex (offset), 0x42 (segmento). Verificación: $0x0F \ll 2 = 0x3C$ ✓, $0x10 \ll 2 = 0x40$ ✓.

Como el enunciado no da direcciones físicas absolutas para estas rutinas (solo pide instalarlas “en el segmento 0x5500”), el desplazamiento de cada una se obtiene con el operador `OFFSET` del ensamblador, asumiendo que el módulo se ensambló con su segmento de código ubicado en 0x5500 (`ASSUME CS:CodigoPWM` con `CodigoPWM` en esa dirección de segmento):

```
cli
mov ax, 0
mov es, ax
mov word ptr es:[0x3C], offset flanco    ; interrupción 15 = flanco()
mov word ptr es:[0x3E], 0x5500
mov word ptr es:[0x40], offset timer     ; interrupción 16 = timer()
mov word ptr es:[0x42], 0x5500
sti
```

RUTINAS EN ASSEMBLER 8086

```

contando    dw 0
anchoMs     dw 0
bitsAcum    dw 0
byteActual  db 0

PULSO equ 0x200
SALIDA equ 0x201

flanco proc far
    mov word ptr [contando], 1
    mov word ptr [anchoMs], 0
    iret
flanco endp

timer proc far
    push ax
    push bx
    push dx

    cmp word ptr [contando], 0
    je _fin

    inc word ptr [anchoMs]

    mov dx, PULSO
    in al, dx
    and al, 0x01
    jnz _fin                ; el pulso sigue en 1: todavía no terminó

    mov word ptr [contando], 0
    mov ax, word ptr [anchoMs]
    cmp ax, 30
    jae _no_marca

    ; ---- MARCA: cierro el byte actual, relleno con 0 si faltan bits ----
_rellenar:
    mov bx, word ptr [bitsAcum]
    cmp bx, 8
    jge _emitir
    mov al, byte ptr [byteActual]
    shl al, 1
    mov byte ptr [byteActual], al
    inc word ptr [bitsAcum]
    jmp _rellenar
_emitir:
    mov dx, SALIDA
    mov al, byte ptr [byteActual]
    out dx, al

```



```

    mov byte ptr [byteActual], 0
    mov word ptr [bitsAcum], 0
    jmp _fin

_no_marca:
    cmp ax, 40
    jl  _es_cero
    mov bl, 1
    jmp _shiftear
_es_cero:
    mov bl, 0
_shiftear:
    mov al, byte ptr [byteActual]
    shl al, 1
    or  al, bl
    mov byte ptr [byteActual], al
    cmp word ptr [bitsAcum], 8
    jge _fin
    inc word ptr [bitsAcum]

_fin:
    pop dx
    pop bx
    pop ax
    iret
timer endp

```

Verificación PUSH/POP: `flanco` no usa ningún registro de la CPU (solo escribe en memoria vía direccionamiento directo), por lo que no necesita guardar nada — 0 PUSH ↔ 0 POP, balanceado trivialmente. `timer`: push ax,bx,dx ↔ pop dx,bx,ax, orden exactamente inverso, balanceado. Ambas terminan en `iret` (instaladas en el vector).

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
Teoría	Vector de interrupciones: ubicación fija, propósito, instalación, persistencia de valores en memoria global
0	Traducción de comportamiento descrito en prosa a máquina de estados (transiciones + condiciones de guarda)
1	Compilación de subrutinas (PUSH/POP + RET) e instalación de una interrupción (n×4 en el vector)
2	Máquina de estados dentro de una ISR (protocolo con byte de control STX/canal/ETX), cola circular con wraparound, ISR con <code>far</code> / <code>IRET</code>
3	Máquina dedicada: ISR de timer que sondea un sensor y controla actuadores por bits, sin guardado de registros por convención (aunque acá se guardó de todos modos)
4	Doble uso de un mismo timer (contador de sub-eventos de 100ms + contador de segundos hasta un límite), instalación con segmento compartido entre dos interrupciones
5	Medición de ancho de pulso combinando un evento por flanco (hardware) con un timer de alta frecuencia (software) — deserialización bit a bit con relleno/descarte según la posición de la “marca”

Errores reales encontrados y corregidos en el material oficial (4 en total):

#	Ejercicio	Error oficial	Corrección
1	1-B	<code>es:[0x16]</code> / <code>es:[0x18]</code> para interrupción 4	<code>es:[0x10]</code> / <code>es:[0x12]</code> (4×4=16 decimal=0x10 hex, no “0x16”)
2	3-C	<code>es:[0x36]</code> / <code>es:[0x38]</code> para interrupción 9	<code>es:[0x24]</code> / <code>es:[0x26]</code> (9×4=36 decimal=0x24 hex, no “0x36”)
3	2-A	variable declarada <code>carPos</code> pero usada como <code>pos</code> (no compila)	variable unificada a <code>pos</code>
4	2-B	<code>nuevoChar proc</code> (sin <code>far</code>) termina en <code>ret</code>	<code>nuevoChar proc far ... iret</code> (es una ISR instalada en el vector, debe restaurar CS/IP/flags completos)

Para el examen: el error más peligroso de este tema no es de lógica sino de **aritmética**: convertir mal $n \times 4$ de decimal a hexadecimal cuesta puntos aunque toda la rutina esté perfecta. Regla de oro: calculá $n \times 4$ siempre en decimal primero, y recién después convertí ese número a hexadecimal — o, más rápido, corré n en hex y desplazá 2 bits a la izquierda ($n \ll 2$), que da directamente el resultado correcto sin pasar por decimal. Además, antes de dar por buena cualquier rutina de interrupción, contá a mano los PUSH contra los POP (deben ser la misma cantidad, en orden exactamente inverso) y confirmá que termine en `IRET` —nunca `RET`— si la rutina está instalada directamente en el vector de interrupciones.